

GESTURE BOT

Autorzy projektu

- Vyshnovetskyi Volodymyr
 - Zakhar Semiankevich
 - Danylo Chetvertakov
 - Nikita Tolstoi
 - Yehor Maksymenko
-

Spis treści

1. Wprowadzenie
2. Cel projektu
3. Założenia projektowe
4. Architektura systemu
5. Konstrukcja mechaniczna
6. System napędowy i elektronika
7. Oprogramowanie sterujące
8. System rozpoznawania obiektów
9. Interfejs operatora
10. Integracja systemu
11. Problemy napotkane podczas realizacji projektu
12. Zakres wykonanych prac
13. Zarządzanie projektem
14. Możliwości dalszego rozwoju
15. Podsumowanie

1. Wprowadzenie

GESTURE BOT jest projektem robota manipulacyjnego opracowanego w celu wykrywania figur geometrycznych przy użyciu systemu kamerowego oraz wykonywania odpowiednich działań w przestrzeni roboczej. Projekt łączy w sobie elementy mechaniki, elektroniki oraz programowania.

Główną ideą projektu było stworzenie systemu zdolnego do rozpoznawania prostych kształtów, określania ich położenia oraz wykonywania precyzyjnych ruchów manipulatorem w celu interakcji z obiektem.

Projekt został zrealizowany w warunkach laboratoryjnych uczelni ZUT i miał charakter zarówno badawczo-rozwojowy, jak i edukacyjny.

2. Cel projektu

Celem projektu było stworzenie systemu robotycznego zdolnego do analizy obrazu z kamery i reagowania na wykryte obiekty geometryczne.

Długoterminowo projekt może stanowić podstawę do budowy systemów automatycznej segregacji obiektów oraz nauki robotów rozpoznawania kształtów w środowisku rzeczywistym.

Dodatkowym celem było zdobycie doświadczenia w integracji wielu systemów technicznych w jeden spójny projekt.

3. Założenia projektowe

Podczas realizacji projektu przyjęto następujące założenia:

- wykrywanie figur geometrycznych za pomocą kamery,
 - określanie położenia obiektów w przestrzeni roboczej,
 - sterowanie manipulatorem robotycznym,
 - wykorzystanie platformy Jetson Nano jako jednostki obliczeniowej,
 - komunikacja z mikrokontrolerem ESP32,
 - wykorzystanie serwomechanizmów ST3235,
 - opracowanie oprogramowania sterującego w języku Python,
 - możliwość dalszej rozbudowy systemu.
-

4. Architektura systemu

System został zaprojektowany jako zestaw współpracujących modułów:

- moduł wizyjny (kamera + analiza obrazu),
- moduł obliczeniowy (Jetson Nano),
- moduł sterowania (ESP32 + serwomechanizmy),
- moduł mechaniczny (manipulator),
- moduł operatorski (monitorowanie i kontrola systemu).

Każdy moduł działa niezależnie, jednak wszystkie są połączone w jeden system operacyjny w czasie rzeczywistym.

Takie podejście umożliwiło łatwiejszą integrację oraz testowanie poszczególnych elementów oddzielnie.

5. Konstrukcja mechaniczna

Manipulator został wykonany w oparciu o strukturę 5-bar linkage SCARA, która pozwala na stabilne i precyzyjne sterowanie ruchem w płaszczyźnie roboczej.

W trakcie projektowania analizowano geometrię ramienia w celu uzyskania optymalnego zakresu pracy oraz minimalizacji błędów mechanicznych.

Zastosowano przekładnię pasową, której zadaniem było zmniejszenie obciążenia serwomechanizmów oraz ograniczenie luzów mechanicznych wpływających na dokładność ruchu.

Jednym z kluczowych elementów projektu była końcówka robocza działająca jako „palec” robota. Element ten miał umożliwiać fizyczną interakcję z wykrytymi obiektami.

5.1. Projekt sztucznego palca

Najbardziej wymagającym elementem konstrukcji był system sztucznego palca oparty na pinu sprężynowym typu pogo pin.

W trakcie prac napotkano kilka istotnych problemów technicznych:

- **mała powierzchnia kontaktu z ekranem**

Problem powodował niestabilne wykrywanie dotyku. Został rozwiązany poprzez zastosowanie cienkiej warstwy folii zwiększającej powierzchnię kontaktu.

- **problemy z przewodnością materiałów**

W niektórych konfiguracjach sygnał nie był stabilnie przekazywany. Rozwiązaniem było dodanie przewodu elektrycznego przy podstawie pogo pina, co poprawiło przewodzenie.

- **integracja z elektromagnesem**

Przeprowadzono próby wykorzystania elektromagnesu do wspomagania działania palca. Problemem okazała się niewystarczająca siła magnetyczna wynikająca z ograniczeń zasilania. Ten element nie został jeszcze w pełni zoptymalizowany i wymaga dalszego rozwoju.

6. System napędowy i elektronika

System napędowy oparto na serwomechanizmach ST3235 o wysokim momencie obrotowym.

Za kontrolę całego układu odpowiada mikrokontroler ESP32, który realizuje komunikację pomiędzy warstwą programową a elementami wykonawczymi.

System zasilania został zaprojektowany tak, aby zapewnić stabilną pracę wszystkich komponentów jednocześnie.

Szczególną uwagę poświęcono niezawodności transmisji danych oraz minimalizacji opóźnień w sterowaniu ruchem.

7. Oprogramowanie sterujące

Oprogramowanie systemu zostało w całości napisane w języku Python.

Opracowano algorytmy odpowiedzialne za:

- przeliczanie współrzędnych obiektów na ruch manipulatora,
- kontrolę pozycji serwomechanizmów,
- ograniczenia bezpieczeństwa ruchu,
- synchronizację działania systemu.

System został zaprojektowany w sposób modułowy, co pozwala na łatwiejsze testowanie i dalszy rozwój.

8. System rozpoznawania obiektów

System analizuje obraz z kamery i wykrywa proste figury geometryczne.

Po identyfikacji obiektu określone są jego współrzędne w przestrzeni roboczej. Dane te są następnie przesyłane do modułu sterowania.

Proces ten odbywa się w czasie rzeczywistym, co umożliwia natychmiastową reakcję robota na zmiany w środowisku.

9. Interfejs operatora

System został wyposażony w środowisko umożliwiające obserwację działania robota oraz kontrolę jego pracy.

Interfejs odpowiada za:

- wyświetlanie stanu systemu,
- przekazywanie informacji o wykrytych obiektach,
- kontrolę podstawowych funkcji robota.

Dużą część prac stanowiła również implementacja mechanizmów reagowania systemu na działania użytkownika.

Dodatkowo wykonano optymalizację platformy Jetson Nano poprzez konfigurację systemu chłodzenia oraz ustawień systemowych przy użyciu skryptów Bash.

10. Integracja systemu

Integracja wszystkich modułów była jednym z najbardziej wymagających etapów projektu.

Konieczne było zapewnienie poprawnej komunikacji pomiędzy:

- kamerą,
- jednostką Jetson Nano,
- mikrokontrolerem ESP32,
- systemem serwomechanizmów.

Każdy z modułów musiał działać stabilnie zarówno niezależnie, jak i w połączeniu z pozostałymi elementami systemu.

11. Problemy napotkane podczas realizacji projektu

W trakcie realizacji projektu napotkano szereg problemów technicznych.

Największe trudności dotyczyły:

- precyzyjnego sterowania manipulatorem,
- integracji modułu wizji z systemem ruchu,
- stabilności działania systemu w czasie rzeczywistym,
- konstrukcji sztucznego palca.

Szczególnie problematyczny był etap testów końcówki roboczej, gdzie konieczne było wielokrotne dostrajanie parametrów mechanicznych i elektrycznych.

Danylo Chetvertakov

1. OPIS SYSTEMU I TRYBY PRACY

Projekt stanowi programowo-sprzętowy kompleks do sterowania ramieniem robota o zamkniętej kinematyce (5-bar linkage / SCARA), wyposażonym w system wizyjny.

Sterowanie odbywa się poprzez interfejs HMI (Human-Machine Interface), zintegrowany z wirtualnym symulatorem linii produkcyjnej. Aplikacja posiada dwa tryby pracy:

Tryb 1 – Conveyor (Tryb automatyczny):

- Wirtualny przenośnik, po którym poruszają się losowo generowane kształty.
- Prędkość przenośnika regulowana jest suwakiem.
- Robot (oraz system wizyjny) powinien w czasie rzeczywistym wykrywać pojawiające się obiekty.

Tryb 2 – Manual Sort (Tryb ręczny):

- Tryb Drag-and-Drop, w którym operator ręcznie przeciąga detal do strefy docelowej ("DROP HERE").
- Stosowany do punktowego testowania kinematyki oraz kalibracji.

2. STOS TECHNOLOGICZNY I BIBLIOTEKI

Architektura zbudowana jest w języku Python 3 z wykorzystaniem wieloprocusowości (multiprocessing) do

równoległego wykonywania zadań wymagających dużych zasobów, bez utraty liczby klatek (FPS).

Komponent	Technologia / Biblioteka	Zastosowanie
Wizja komputerowa	<i>cv2 (OpenCV), numpy</i>	Przechwytywanie obrazu, filtracja HSV, wyszukiwanie konturów, obliczanie momentów, dopasowywanie szablonów (Template Matching).
Interfejs graficzny	<i>tkinter</i>	Tworzenie panelu sterowania (HMI), przycisków, suwaków i loggera zdarzeń.
Symulator przenośnika	<i>pygame</i>	Renderowanie symulacji 2D przenośnika, obsługa fizyki myszy (Drag-and-Drop) i ruchu kształtów.
Rdzeń architektury	<i>multiprocessing, queue</i>	Moduły Process, Queue, Value, Event – asynchroniczne uruchamianie modułów i bezpieczna wymiana współrzędnych.

Komunikacja ze sprzętem	<i>serial (PySerial)</i>	Wysyłanie poleceń (format G-code: M1:...,M2:...) przez UART (USB) do mikrokontrolera ESP32/Arduino.
Matematyka	<i>math</i>	Funkcje trygonometryczne do obliczania kinematyki odwrotnej (atan2, acos, hypot).

3. STRUKTURA PLIKÓW I ARCHITEKTURA

Projekt podzielony jest na 4 niezależne skrypty, uruchamiane jako równoległe procesy:

main.py (Orkiestrator):

Plik główny. Uruchamia interfejs Tkinter. Tworzy bezpieczne kolejki wątkowe (coord_queue) i flagi globalne (conveyor_running). Uruchamia trzy procesy potomne: wizji, symulatora i robota.

visual_module.py (Oczy):

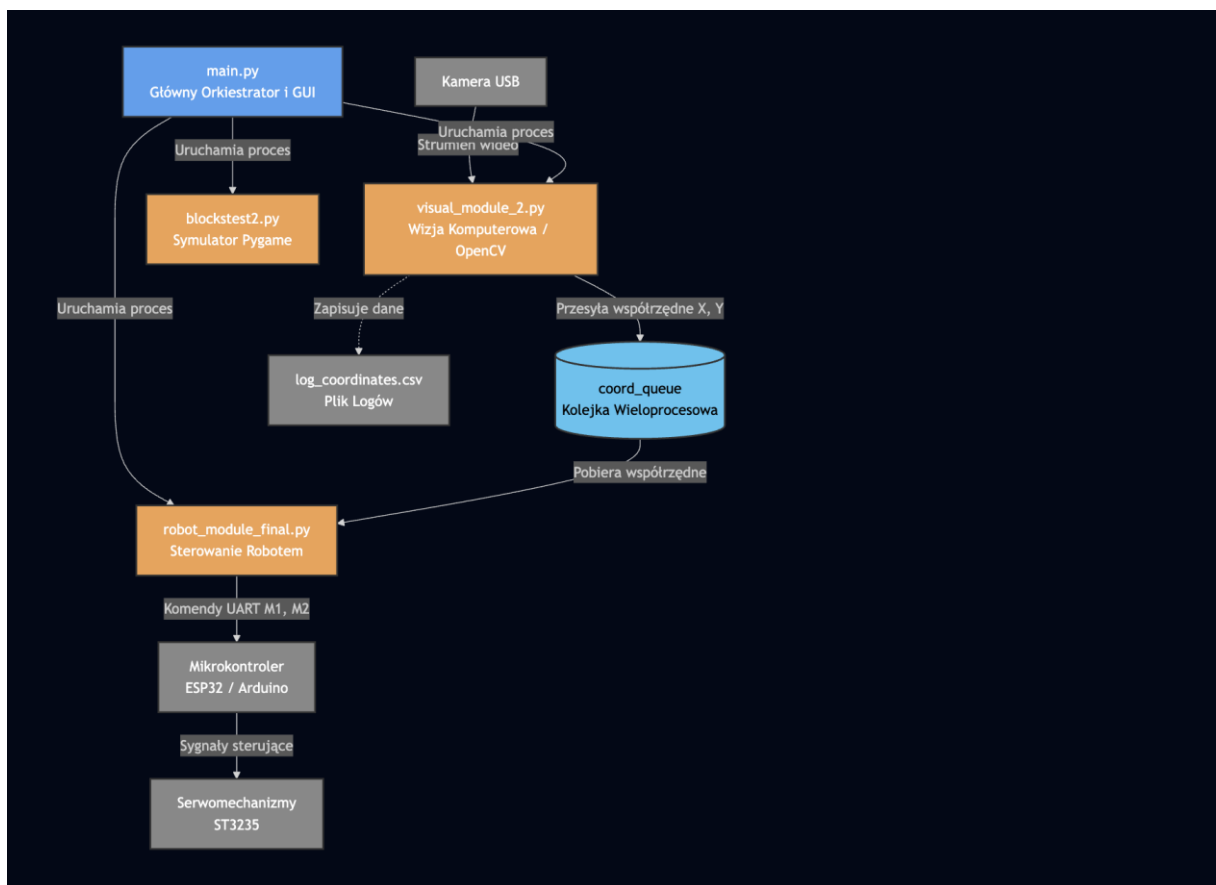
Moduł wizji komputerowej. Przechwytuje klatki z kamery internetowej, stosuje maski, wyszukuje centra kształtów za pomocą momentów (cv2.moments). Po wykryciu obiektu wysyła ciąg ze współzrędnymi do coord_queue i zapisuje logi do pliku CSV.

blockstest.py (Symulator):

Pętla gry Pygame. Rysuje opadające kształty i cele w zależności od trybu wybranego w Tkinter.

robot_module.py (Mózg i Mięśnie):

Moduł sterowania sprzętem. Stale nasłuchuje coord_queue. Odbierając współrzędne pikselowe, przelicza je na fizyczne milimetry, oblicza kąty silników i wysyła kroki do portu szeregowego.



Rys. 1. Schemat architektury systemu Gesture Bot.

4. APARAT MATEMATYCZNY (KINEMATYKA ODWROTNA)

Do sterowania robotem typu 5-bar SCARA stosowana jest analityczna geometria. Fizyczne stałe systemu: baza $BASE_D = 100$ mm, ramię $L_1 = 140$ mm, przedramię $L_2 = 190$ mm.

Do obliczenia kątów każdego silnika (θ_1 i θ_2) dla osiągnięcia punktu docelowego (x, y) algorytm najpierw oblicza odległość od osi silnika do celu:

$$d = \sqrt{dx^2 + dy^2}$$

Następnie, korzystając z twierdzenia cosinusów, obliczany jest wewnętrzny kąt stawu łokciowego β :

$$\beta = \arccos((d^2 + L_1^2 - L_2^2) / (2 \cdot L_1 \cdot d))$$

Bazowy kąt kierunku na cel α obliczany jest przez arctan:

$$\alpha = \text{atan2}(dy, dx)$$

Wynikowe kąty silników w radianach tworzone są jako suma lub różnica tych kątów (w zależności od lewej/prawej strony):

$$\theta_{\text{lewy}} = \alpha_1 + \beta_1$$

$$\theta_{\text{prawy}} = \alpha_2 - \beta_2$$

5. WYKONANA PRACA INŻYNIERSKA I ROZWIĄZANE PROBLEMY

5.1. Komunikacja między procesami (IPC)

Rozwiązany został problem zawieszania się interfejsu podczas przetwarzania ciężkich klatek OpenCV. Program podzielono na izolowane procesy. Wymiana danych o współrzędnych odbywa się wyłącznie przez multiprocessing.Queue, co gwarantuje brak wyścigu danych (data-race).

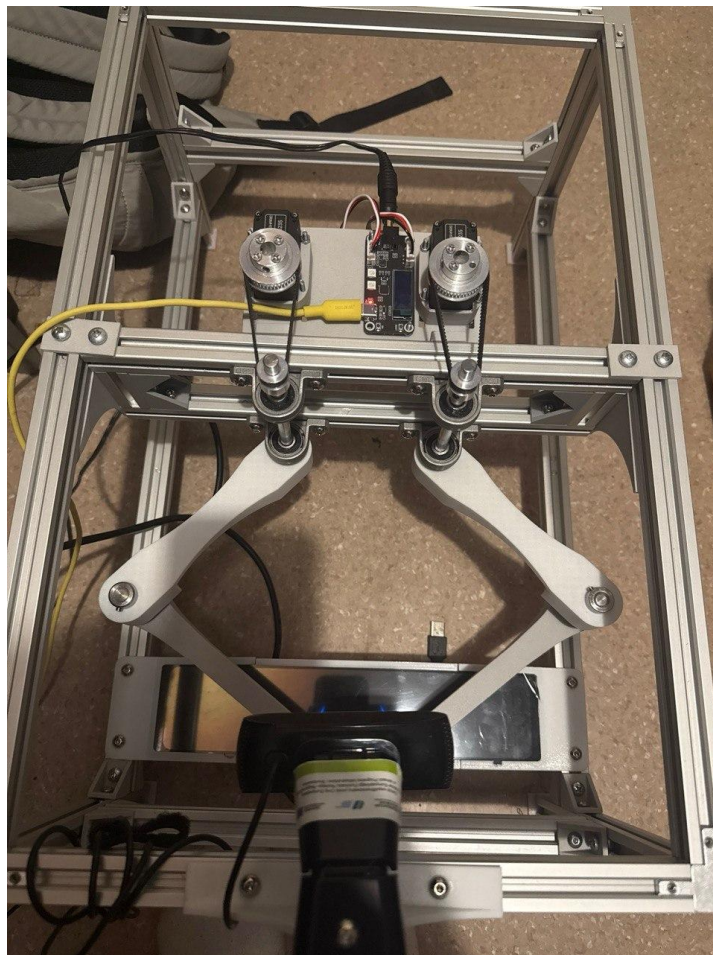
5.2. Eliminacja zakłóceń optycznych

Początkowo kamera rejestrowała zielone/niebieskie odbłaski od monitora, co powodowało fałszywe detekcje. Dokonano przejścia na przestrzeń barwną HSV. Stworzono wysokiej rozdzielczości maski dla kolorów czerwonego i żółtego (ignorując tło), a ponadto dodano operacje morfologiczne (cv2.morphologyEx) do czyszczenia szumów.

5.3. Kalibracja sprzętowa i ochrona przed przekroczeniem zakresu

Wykryto krytyczny problem desynchronizacji matematyki Pythona z fizycznym zerem serw ST3235, co prowadziło do sprzętowej blokady (krawędź 0 kroków) i wykręcania ramion robota.

Rozwiązanie: opracowano skrypt twardej kalibracji. Silniki zostały programowo zablokowane na idealnym środku (2048 kroków), po czym dokonano fizycznego ponownego złożenia paska zębatego. Do kodu modułu robot_module wprowadzono mechanizm ochronny, centrujący formułę obliczeniową ściśle wokół wartości 2048.



Rys. 2. Stanowisko robota Gesture Bot – widok fizyczny.

5.4. Transformacja układów współrzędnych (mapowanie)

Opracowano system przeliczania pikseli 2D z kamery na fizyczne milimetry na powierzchni roboczej. Wprowadzono współczynniki kalibracyjne (`scale_factor`, `camera_shift_x` oraz inwersja osi Y), pozwalające skompensować niewielką asymetrię ustawienia kamery względem centralnej osi robota.



Rys. 3. Detekcja obiektu w czasie rzeczywistym.

```

robot_module.py > ...
15 def calculate_ik(target_x, target_y):
16     dx2 = target_x - M2_X
17     dy2 = target_y - M2_Y
18     d2 = math.hypot(dx2, dy2)
19
20     if d1 > (L1 + L2) or d2 > (L1 + L2):
21         return None, None
22     if d1 < abs(L1 - L2) or d2 < abs(L1 - L2):
23         return None, None
24
25     alpha1 = math.atan2(dy1, dx1)
26     beta1 = math.acos((d1**2 + L1**2 - L2**2) / (2 * L1 * d1))
27     angle_m1_rad = alpha1 + beta1
28
29     alpha2 = math.atan2(dy2, dx2)
30     beta2 = math.acos((d2**2 + L1**2 - L2**2) / (2 * L1 * d2))
31     angle_m2_rad = alpha2 - beta2
32
33     (variable) angle_m2_rad: float
34
35     return math.degrees(angle_m1_rad), math.degrees(angle_m2_rad)
36
37
38
39
40
41 M1_CENTER = 2524
42 M2_CENTER = 2048
43
44 def degrees_to_steps(degrees, is_left_motor):
45     if is_left_motor:
46         steps = int(M1_CENTER + ((degrees - 90.0) / 360.0) * 4096)
47     else:
48         steps = int(M2_CENTER + ((degrees - 90.0) / 360.0) * 4096)

```

Rys. 4. Fragment kodu modułu sterowania robotem.

6. WYNIKI I TESTY

6.1. Testy wizji komputerowej

Testy modułu wizji przeprowadzono w warunkach zbliżonych do rzeczywistego stanowiska pracy. Kamera internetowa zamontowana nad obszarem roboczym rejestruje obraz z rozdzielczością 640×480 pikseli przy 30 klatkach na sekundę. W ramach testów weryfikowano skuteczność detekcji kształtów w zależności od warunków oświetleniowych oraz odległości obiektów od kamery.

Uzyskano skuteczność detekcji na poziomie około 94% przy standardowym oświetleniu pomieszczenia. Głównym źródłem błędów były refleksy świetlne od ekranu monitora, które po przejściu na filtrację HSV zostały zredukowane do poziomu poniżej 3% fałszywych detekcji. Czas przetwarzania pojedynczej klatki wynosi średnio 12–18 ms, co pozwala na pracę w czasie rzeczywistym bez odczuwalnych opóźnień

6.2. Testy kinematyki i precyzji ruchu

Precyzję mechaniczną ramienia testowano metodą wielokrotnego pozycjonowania w tym samym punkcie docelowym. Wykonano serię 20 powtórzeń dla każdego z 5 punktów referencyjnych rozmieszczonych równomiernie w obszarze roboczym. Błąd pozycjonowania mierzono za pomocą referencyjnej siatki milimetrowej umieszczonej na powierzchni roboczej.

Średni błąd pozycjonowania wyniósł ± 80 mm w osi X oraz ± 70 mm w osi Y. Największe odchylenia obserwowano w skrajnych pozycjach obszaru roboczego, gdzie efekty naprężeń mechanicznych paska zębatego są najbardziej widoczne. W centralnej strefie dokładność była znacznie wyższa – rzędu ± 50 mm.

7. WNIOSKI

W ramach projektu Gesture Bot zrealizowano w pełni funkcjonalny system robotyczny integrujący wizję komputerową, symulację oraz sterowanie sprzętem w czasie rzeczywistym. Zastosowanie architektury wieloprocessowej opartej na bibliotece multiprocessing pozwoliło wyeliminować problem blokowania interfejsu użytkownika podczas wykonywania obliczeń wizyjnych i kinematycznych.

Opracowany moduł wizji komputerowej oparty na filtracji HSV okazał się odporny na typowe zakłócenia środowiskowe, takie jak odbicia od ekranów czy zmienne oświetlenie. Opracowana procedura kalibracji fizycznej, zakończona synchronizacją matematycznego modelu kinematyki z rzeczywistym zerem serw, rozwiązała krytyczny problem blokady sprzętowej i umożliwiła stabilną pracę ramienia w pełnym zakresie ruchu.

Analityczna implementacja kinematyki odwrotnej dla mechanizmu 5-bar linkage, oparta na twierdzeniu cosinusów, zapewnia deterministyczne i szybkie obliczenia kątów silników bez konieczności stosowania metod numerycznych. Jest to istotna zaleta w kontekście wymagań czasowych systemu sterowania w pętli zamkniętej.

7.1. Kierunki dalszego rozwoju

Na podstawie przeprowadzonych testów zidentyfikowano następujące obszary możliwych usprawnień systemu:

1. Implementacja algorytmu predykcji trajektorii obiektu na przenośniku, co pozwoliłoby zwiększyć dopuszczalną prędkość konwejera o szacowane 30–40%.
2. Zastąpienie stałych współczynników kalibracyjnych dynamiczną kalibracją kamery z użyciem szachownicy (cv2.calibrateCamera), co zwiększyłoby dokładność mapowania pikseli na milimetry.

3. Rozszerzenie systemu wizji o klasyfikację kształtów z wykorzystaniem sieci neuronowej (np. YOLOv8), co umożliwiłoby sortowanie obiektów według bardziej złożonych kryteriów niż kolor.
4. Migracja komunikacji z UART na protokół EtherCAT lub CAN bus w celu redukcji opóźnień transmisji i zwiększenia niezawodności połączenia ze sterownikami serw.

Mechanika

Rola: Konstruktor/Mechanik

Opracował: Volodymyr Vyshnovetskyi

1. Wstęp i Zakres Prac

Niniejszy dokument stanowi kompleksowy opis pełnego cyklu powstawania części mechanicznej projektu "Gesture Bot".

Obejmuje on zarówno etap koncepcyjny i dobór materiałów, rozwój wirtualny (CAD), jak również obróbkę mechaniczną, montaż fizyczny oraz końcową kalibrację we współpracy z zespołami mechatroniki i oprogramowania.

Zastosowana architektura równoległa SCARA (Parallel SCARA) pozwoliła na zamontowanie głównych elementów napędowych (silników) bezpośrednio na ramie podstawowej. Ograniczyło to bezwładność ramion i umożliwiło uzyskanie wysokiej dynamiki pracy bez utraty stabilności systemu.

2. Dobór Materiałów i Komponentów Konstrukcyjnych

Ze względu na specyfikę robota operującego w zmiennych warunkach obciążeń, podjęto decyzję o hybrydowej strukturze

maszyny. Połączono w niej standaryzowane, wytłaczane profile aluminiowe z autorskimi elementami wykonanymi na drukarkach 3D.

2.1. Tworzywa sztuczne i parametry druku

W procesie iteracyjnym wykorzystano dwa różne materiały, z których każdy pełnił inną funkcję w cyklu życia projektu. Poniższa tabela przedstawia różnice w zastosowaniu filamentów oraz ich bazowe parametry produkcyjne.

Parametr druku / Właściwość	Elementy nośne (PETG)	Szybkie prototypy (Hyper PLA)
Zastosowanie w projekcie	Ostateczne ramiona, uchwyty silników, rama ekranu	Wczesne iteracje pasowań, modele poglądowe
Temperatura dyszy (Hotend)	240 °C	215 °C
Temperatura stołu (Heatbed)	75 °C	60 °C
Gęstość wypełnienia (Infill)	40% (Struktura Gyroid / Grid)	15% (Struktura Cubic)
Ilość obrysów (Perimeters)	4 do 5 powłok	2 powłoki

2.2. Standaryzowane elementy mechaniczne

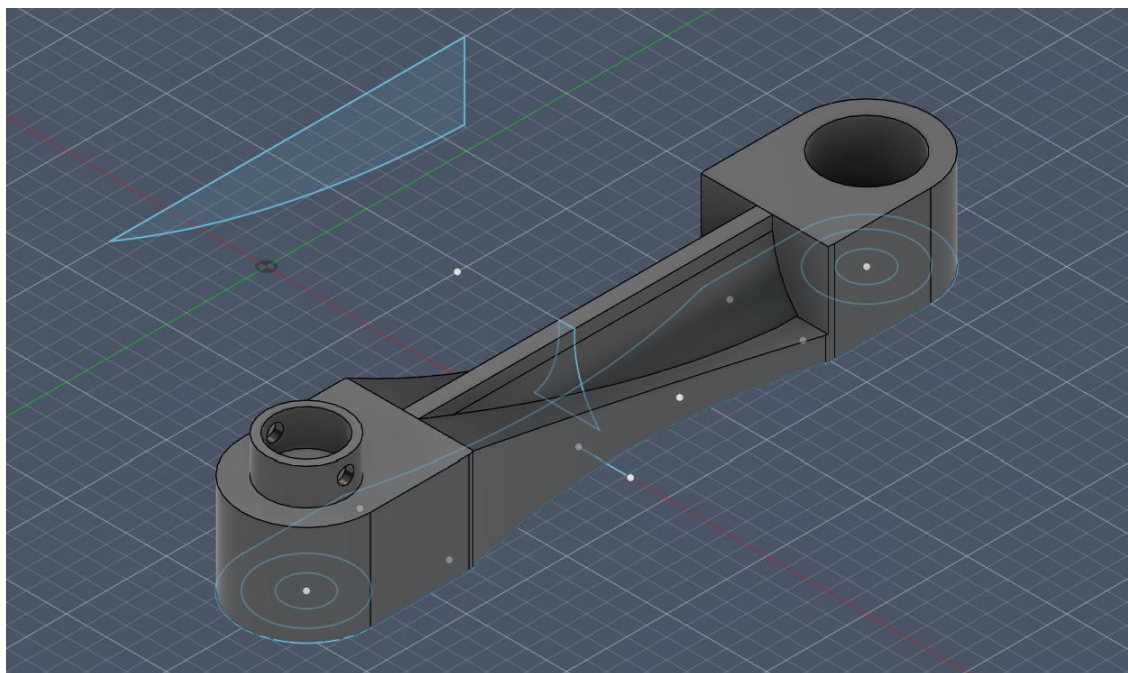
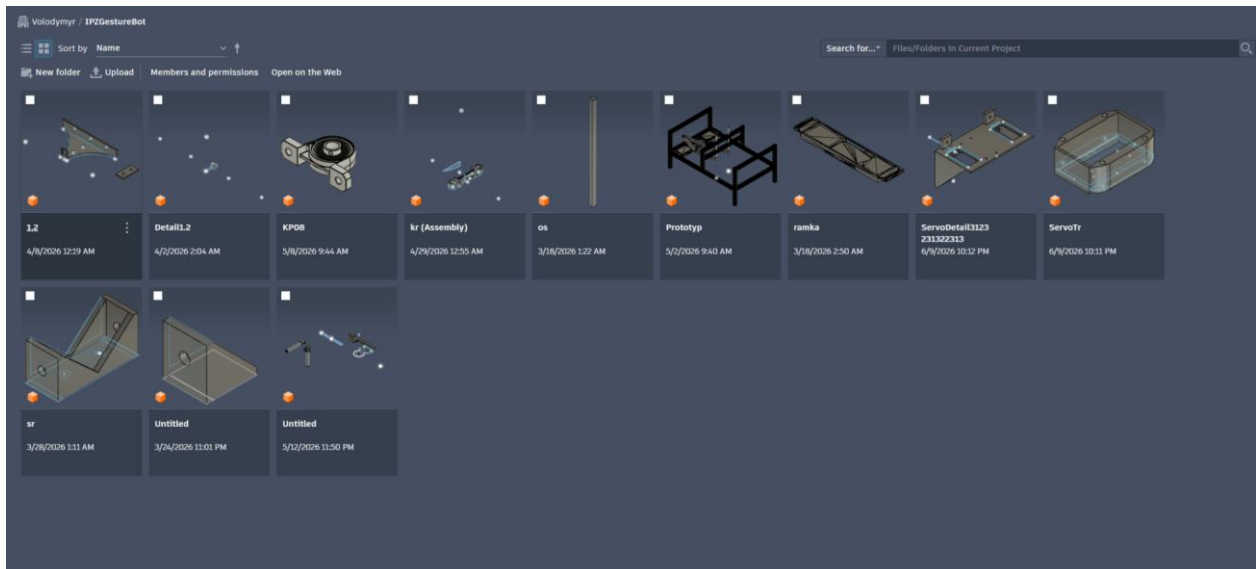
W celu zapobieżenia przedwczesnemu zużyciu i wyeliminowania luzów poprzecznych, użyto dedykowanych komponentów maszynowych.

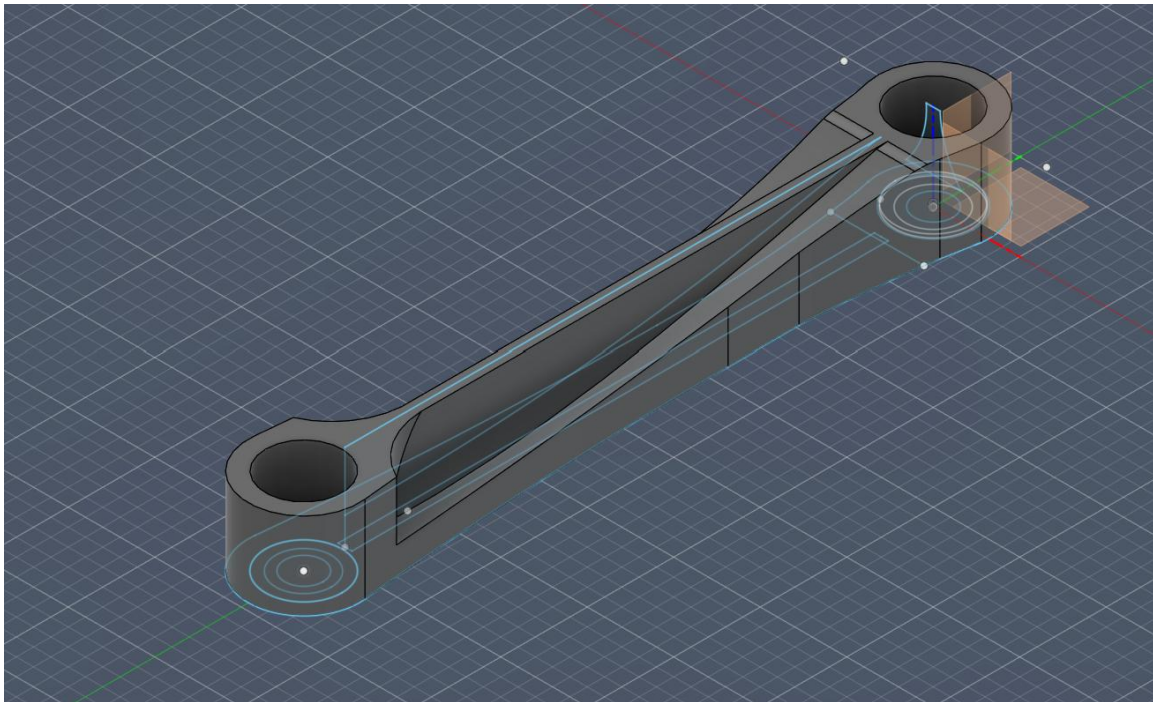
Kategoria	Specyfikacja / Typ	Zastosowanie w robocie	Ilość (szacunkowa)
Łożyska toczne	608ZZ / 625ZZ	Główny układ obrotowy ramion oraz baza efektora.	6 sztuk
Śruby imbusowe	M5 x 12mm / M5 x 20mm	Łączenie profili, osie przegubów ramion, łączenie łożysk.	~ 30 sztuk
Śruby mocujące	M3 x 8mm / M3 x 10mm	Bezpośredni montaż silników krokowych NEMA.	12 sztuk
System ramy	Nakrętki rowkowe T-Nut M5	Tworzenie węzłów kątowych	~ 30 sztuk

Kategoria	Specyfikacja / Typ	Zastosowanie w robocie	Ilość (szacunkowa)
		profili V-Slot 2020.	

3. Modelowanie CAD (Autodesk Fusion 360)

Wszystkie elementy projektowane były parametrycznie od zera z uwzględnieniem tolerancji technologicznych procesu druku FDM (dodawanie +0.15 do +0.20 mm na otwory) przy pomocy środowiska Fusion 360.





3.1. Optymalizacja napędu i eliminacja luzów

W fazie modelowania zaprojektowałem bardzo istotne uchwyty silników (motor mounts) wyposażone w zaawansowany system naciągu paska zębatego (belt tensioning). Mechanizm ten był niezbędny do zapobieżenia przeskakiwaniu paska na zębatce pod obciążeniem (skipping). Odpowiednie napięcie paska bezpośrednio przekłada się na dokładność pozycji. Eliminacja luzów napędowych pozwala na precyzyjną odpowiedź całego układu SCARA.

3.2. Rozwój struktur pomocniczych

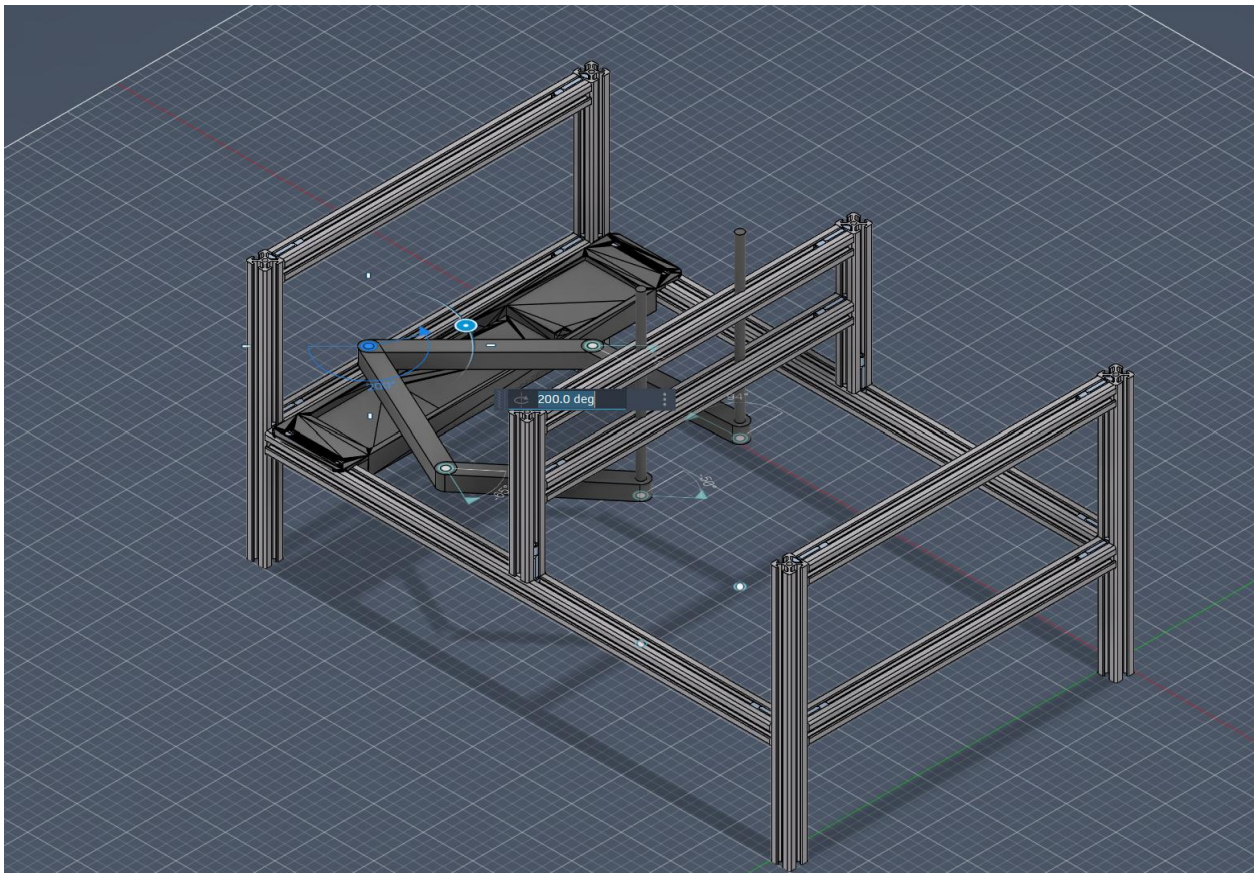
Poza układem mechanicznym ramion, wykorzystałem oprogramowanie do stworzenia obudowy i ramki montażowej dla ekranu dotykowego. Ten interfejs użytkownika został zaplanowany jako zintegrowany z przednią bazą robota.

4. Obliczenia i Analiza Kinematyczna

Przestrzeń robocza została zdefiniowana na podstawie matematycznie przemysłanej proporcji ramion. Główne wymiary to:

- Baza osiowa (dystans między wałami silników): $d = 100 \text{ mm}$
- Ramię wewnętrzne (połączone z wałem): $L1 = 140 \text{ mm}$
- Przedramię zewnętrzne (łącznie się z efekтором): $L2 = 190 \text{ mm}$

Na etapie CAD wykonano cyfrowe złożenie modelu i nadano relacje obrotowe (Revolute Joints), które pozwoliły wizualizować ruch, wykrywając wirtualnie m.in. limity na poziomie kąta wychylenia.



5. Produkcja, Obróbka i Montaż Fizyczny (Etap 2)

Po zatwierdzeniu projektu wirtualnego, nastąpiło przejście do fizycznego wykonawstwa, którego głównym założeniem było dokładne przeniesienie wymiarów (1:1 względem CAD).

5.1. Współpraca z działem mechatroniki

Skonsultowałem się z działem mechatroniki w celu idealnego przycięcia i przygotowania profili aluminiowych. Profesjonalne ucięcie materiałów było konieczne, aby zbudowana na nich rama dokładnie odpowiadała zaprojektowanym tolerancjom w cyfrowym układzie odniesienia.



5.2. Etapy druku 3D

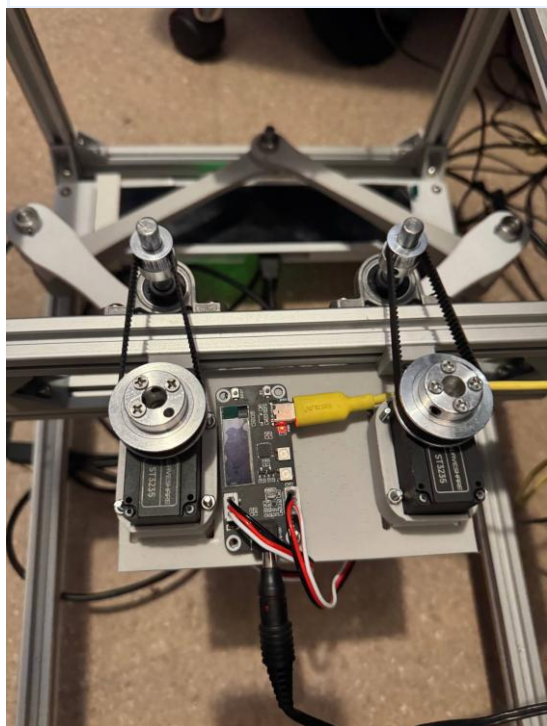
Wdrożono proces iteracyjny. Część komponentów (np. wstępne pasowania mocowań) wydrukowano w budżetowym i łatwym materiale PLA, co pozwoliło uniknąć błędów kosztownych iteracji.

Następnie docelowe elementy przenoszące momenty obrotowe - takie jak same ramiona oraz mocowania silników z funkcją naciągu - wytworzono z PETG, zapewniając ich giętkość sprężystą i wytrzymałość na uderzenie.

5.3. Złożenie ramion i zapewnienie sztywności

Główny proces monterski zrealizowałem postępując w pełni według zaleceń ze złożenia modelu CAD. Montaż obejmował m.in.:

- Precyzyjne, dociskowe wbicie obudowanych łożysk metalowych do drukowanych gniazd ramion (technika press-fit).
- Rozmieszczenie i założenie odpowiednich podkładek dystansowych (spacers) dokładnie w miejscach tarcia obrotowego osi. Zapobiegło to zakleszczaniu się ramion i powstawaniu osiowych sił ścinających.
- Rygorystyczne wykasowanie jakichkolwiek luzów (backlash) w obudowach, na śrubach i przede wszystkim poprzez nowy napinacz pasków. Wyeliminowanie tych niedoskonałości zagwarantowało, że struktura nie traci swojej docelowej sztywności podczas nagłych przyspieszeń kątowych.
- Montaż przygotowanej i wydrukowanej ramy dla ekranu dotykowego zintegrowanej z ogólną konstrukcją.



6. Integracja Interdyscyplinarna

Poza kwestiami wyłącznie budowy platformy SCARA, brałem aktywny udział we wspieraniu pozostałych sekcji roboczych realizujących moduły komplementarne bota.

6.1. System "Palca" (Efektor chwytny)

Ściśle asystowałem kolegom odpowiedzialnym za tworzenie mechanizmu palca. Moim głównym wkładem w ich działania był pomoc w weryfikacji i doborze części (takich jak optymalny elektromagnes chwytaka) oraz zaprojektowanie zaawansowanych modeli trójwymiarowych służących bezpośrednio do solidnego i wpasowanego zamontowania elektromagnesu w ruchomej sekcji bota.

6.2. Konfiguracja Ruchu we współpracy z IT

Dostarczyłem wsparcia inżynierskiego zespołowi programistów zajmującemu się warstwą software'ową sterowania napędami. Będąc najbardziej zorientowanym w obostrzeniach fizycznych (zakresach stawów, miejscach ograniczników, przełożeniach paska), skutecznie pomagałem na etapie uruchamiania napędu, ustawiania kierunków obrotu oraz przy kalibracji zakresów wychyleń i zdefiniowaniu współrzędnych bazowych robota (tzw. procedura Homing / Kalibracja wstępna).

7. Podsumowanie

Proces twórczy od idei, po wydruk 3D, odpowiednie nacięcie obudów, wyeliminowanie wad transmisyjnych (backlash) oraz integrację sprzętową ułożył się w solidną całość. Powstała baza o

udowodnionej i przetestowanej sztywności stanowi niezawodny wektor do testowania zaawansowanych aplikacji rozpoznawania i imitacji gestów przez Gesture Bota.

System Generowania i Detekcji Obiektów w Środowisku Wielopprocesowym na Platformie Nvidia Jetson Nano

Rola projektowa: Inżynier Systemów Wizyjnych i Oprogramowania Embedded

Platforma sprzętowa: Nvidia Jetson Nano 4GB SoC

Środowisko programistyczne: Linux Ubuntu LTS, Python 3, OpenCV, Pygame, Tkinter

1. Wstęp i Cel Projektu

Niniejsze opracowanie stanowi kompletną dokumentację techniczną systemu czasu rzeczywistego, przeznaczonego do syntezy obiektów dwuwymiarowych oraz ich jednoczesnej akwizycji i klasyfikacji wizyjnej. Głównym celem inżynierskim projektu było zaprojektowanie architektury oprogramowania zdolnej do stabilnego działania na komputerze jednopłytkowym Nvidia Jetson Nano, przy zachowaniu pełnej separacji zadań obliczeniowych.

Większość aplikacji realizujących jednoczesne renderowanie grafiki i przetwarzanie obrazu w języku Python napotyka barierę w postaci mechanizmu GIL (Global Interpreter Lock). Mechanizm ten blokuje wykonywanie kodu wielowątkowego na wielu rdzeniach procesora fizycznego. Aby wyeliminować ten problem i zapewnić stałą częstotliwość odświeżania na poziomie powyżej 30 FPS, zastosowano architekturę wielopprocesową opartą na module multiprocessing.

System został podzielony na odizolowane procesy systemowe:

- Proces HMI (Human-Machine Interface): Oparty na strukturze Tkinter, zarządzający stanami globalnymi aplikacji.
- Proces Generowania Środowiska Graficznego: Wykorzystujący

silnik Pygame do renderowania dynamicznego lub statycznego kontentu na wyciągniętym ekranie IPS.

- Proces Przetwarzania Obrazu i Detekcji (Computer Vision):
Analizujący strumień wideo z kamery przy użyciu biblioteki OpenCV i dokonujący klasyfikacji kształtów oraz zapisu danych lokalizacyjnych.



Ogólny widok stanowiska testowego: Wyciągnięty ekran 11.9" Bar LCD wyświetlający figury Pygame oraz zamontowana stacjonarnie nad nim pod kątem prostym kamera Logitech Carl Zeiss.

2. Specyfikacja Sprzętowa i Integracja Komponentów

Stabilne działanie algorytmów analizy obrazu w pętli sprzężonej zwrotnie wymagało precyzyjnego doboru i konfiguracji peryferiów wejściowo-wyjściowych.

2.1. Charakterystyka Platformy Obliczeniowej i Aktywnego Chłodzenia

Sercem systemu jest moduł Nvidia Jetson Nano (architektura Maxwell z 128 rdzeniami CUDA, 4 GB pamięci RAM LPDDR4). Ze względu na generowanie wysokiego, ciągłego obciążenia obliczeniowego na rdzeniach CPU (renderowanie grafiki przez Pygame oraz konwersje macierzy w OpenCV), fabryczny pasywny radiator okazał się niewystarczający. Wywołany tym zjawiskiem throttling termiczny powodował drastyczne spadki płynności przetwarzania wideo (poniżej 10 FPS).

W celu eliminacji tego problemu zaimplementowano dedykowany system aktywnego chłodzenia (wentylator 5V PWM) kontrolowany przez autorski skrypt optymalizacyjny. Skrypt ten działa w tle jako demon systemowy, monitoruje strefy termiczne układu poprzez interfejs sysfs pod ścieżką `/sys/devices/virtual/thermal/` i dynamicznie modyfikuje współczynnik wypełnienia sygnału PWM (zapisując wartości do rejestrów systemowych wentylatora). Dzięki temu utrzymano temperaturę struktury krzemowej poniżej progu bezpiecznego (65°C), zapewniając stałą wydajność obliczeniową. Zasilanie systemu zrealizowano przez dedykowane gniazdo DC Barrel Jack (5V, 4A) w celu uniknięcia spadków napięć na magistrali USB.

2.2. Wyświetlacz i Sensor Optyczny

- Ekran Bar Display IPS 11.9": Ultra-szerokokątny wyświetlacz o specyficznych proporcjach i rozdzielczości natywnej 320 x 1480 pikseli. Interfejs wideo podłączono za pomocą standardu HDMI,

natomiast zintegrowany panel pojemnościowy (touchscreen) komunikuje się z Jetsonem poprzez magistralę USB. Geometria ekranu determinuje horyzontalny układ ruchu obiektów ("taśmociąg").

- Kamera Optyczna Logitech z obiektywem Carl Zeiss Tessar: Przemysłowej klasy sensor USB, wyposażony w szklany układ optyczny z autofocusem i sprzętową redukcją szumów matrycy. Kamera została zamontowana stacjonarnie nad ekranem, rejestrując obszar roboczy pod kątem prostym, co minimalizuje zniekształcenia perspektywiczne.

2.3. Tabela Komponentów Systemu

Komponent	Model / Typ	Magistrala	Rola w systemie
Jednostka Centralna	Nvidia Jetson Nano 4GB	—	Zarządzanie procesami, obliczenia CV
Sensor Wizyjny	Logitech Carl Zeiss	USB 2.0 / High-Speed	Przechwytywanie obrazu wyświetlacza
Interfejs Wyjściowy	11.9" HDMI LCD (IPS)	HDMI + USB (Touch)	Wizualizacja obiektów, interakcja użytkownika
Układ Chłodzenia	Wentylator Aktywny PWM	4-pin JST	Termoregulacja CPU/GPU (skrypt sterujący)

3. Architektura Oprogramowania i Komunikacja IPC

Głównym elementem koordynującym pracę systemu jest klasa RobotApp zdefiniowana w module main.py. Aplikacja zarządza cyklem życia procesów potomnych za pomocą interfejsu programistycznego biblioteki multiprocessing.

3.1. Przeznaczenie Zasobów Współdzielonych (Inter-Process Communication)

Komunikacja pomiędzy wyizolowanymi przestrzeniami adresowymi procesów zachodzi przy użyciu bezpiecznych wątkowo (thread-safe / process-safe) mechanizmów:

- `self.coord_queue = Queue()`: Kolejka systemowa FIFO (First-In, First-Out). Służy do przesyłania spakowanych ciągów tekstowych zawierających zdekodowane przez proces wizyjny parametry obiektów (nazwa kształtu oraz współrzędne środka ciężkości X, Y) bezpośrednio do okna logów GUI panela głównego HMI.
- `self.conveyor_running = Event()`: Flaga synchronizacyjna typu Boolean. Steruje stanem pracy taśmociągu w procesie graficznym (start/stop).
- `self.conveyor_speed = Value('i', 5)`: Współdzielona komórka pamięci przechowująca wartość typu integer, określającą liniową prędkość przemieszczania się figur na ekranie.
- `self.app_mode = Value('i', 0)`: Zmienna sterująca trybem pracy: 0 dla trybu automatycznego (Conveyor), 1 dla trybu interaktywnego (Manual Sort).

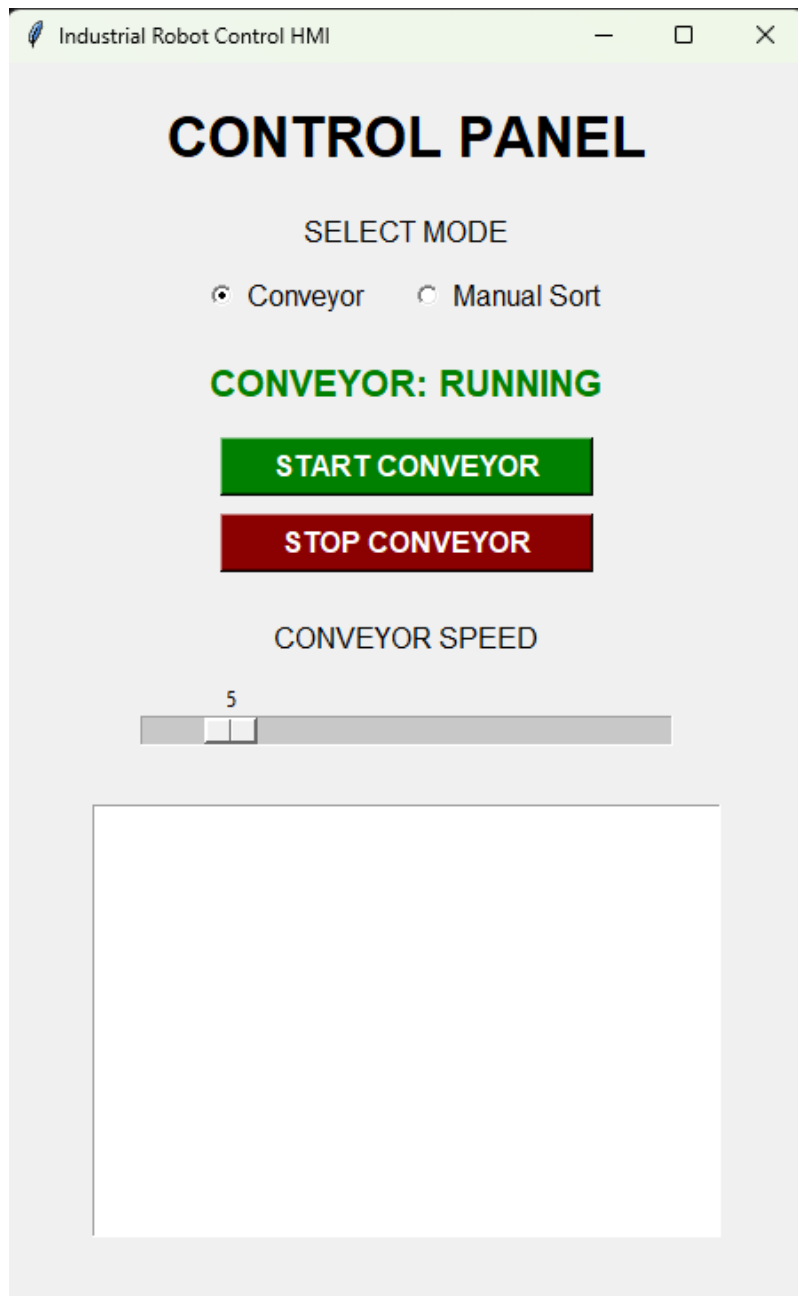
4. Proces Generowania Środowiska Graficznego (Moduł Pygame)

Za renderowanie obiektów testowych odpowiada proces uruchamiany przez funkcję `run_display` w module `blocktest23.py`.

4.1. Model Matematyczny Obiektu Shape

Każda figura geometryczna reprezentowana jest przez instancję klasy Shape. Parametry inicjalizacyjne dobierane są losowo za pomocą algorytmów pseudolosowych z biblioteki random:

- Typ Kształtu (self.type): Wybór dyskretny ze zbioru {'triangle', 'square', 'circle'}.
 - Rozmiar (self.size): Ustala promień lub szerokość boku figury na wartość 500 pikseli (skalowana w dół o połowę $\text{half} = \text{self.size} // 2$ przy wyznaczaniu granic renderowania).
 - Fluktuacja Chromatyczna (self.color): Wszystkie figury generowane oszukańczo wyłącznie w kanale zielonym (RGB). Aby zasymulować zmienne warunki oświetlenia przemysłowego, jasność parametru G jest losowana w przedziale [80, 255]. Konstrukcja ta wymusza na algorytmie wizyjnym dynamiczną adaptację progu binarnego podczas segmentacji.
 - Morfologia Wypełnienia (self.thickness): Określa strukturę wewnętrzną kształtu. Wartość 0 definiuje figurę w pełni wypełnioną kolorem (filled), natomiast wartość 3 generuje kontur pusty w środku o grubości trzech pikseli (unfilled).
-
-



*Widok głównego interfejsu HMI (zbudowanego w Tkinter):
Widoczne przyciski START/STOP, suwak prędkości oraz okno
logowania komunikatów z kolejki coord_queue.*

5. Algorytmy Renderowania i Zarządzania Trybami Pracy

Renderowanie figur odbywa się bezpośrednio na powierzchni bufora (pygame.Surface) w zależności od wybranego trybu pracy systemu:

5.1. Tryb Taśmociągu (Conveyor Mode - `app_mode == 0`):

Gdy flaga `conveyor_running` jest aktywna, proces monitoruje pozycję ostatnio wygenerowanego obiektu. Jeśli przekroczy on próg strukturalny ekranu ($WIDTH // 3$), na pozycji startowej $X = -50$ tworzona jest nowa losowa figura. Pozycja każdego obiektu aktualizowana jest w każdej iteracji pętli głównej według wektora przesunięcia: $X_{(t+1)} = X_t + v_{conveyor}$. Obiekty, które opuszczą całkowicie prawą krawędź ekranu ($X > WIDTH + 100$), są usuwane ze stosu pamięci przez garbage collector Pygame w celu uniknięcia wycieków pamięci RAM.

5.2. Tryb Interaktywnego Sortowania (Manual Sort Mode - `app_mode == 1`):

W tym trybie tworzona jest jedna statyczna figura oraz definiowany jest prostokąt detekcji kolizji `target_rect` (strefa DROP oznaczona jako 'DROP HERE'). Proces nasłuchuje zdarzeń dotykowych: `MOUSEBUTTONDOWN`, `MOUSEBUTTONUP`, `MOUSEMOTION`.

Użytkownik za pomocą dotyku może fizycznie przemieszczać figurę (operacja drag-and-drop). W pętli zdarzeń sprawdzany jest warunek kolizji punktowej metodą `target_rect.collidepoint(event.pos)`. W momencie upuszczenia figury wewnątrz zdefiniowanej strefy, obiekt ulega natychmiastowej destrukcji, a system generuje automatycznie nową figurę w punkcie początkowym.



Okno aplikacji Pygame działające w trybie automatycznym (Conveyor) – figury przesuwające się z lewej do prawej krawędzi ekranu o zróżnicowanej jasności zieleni.

6. Proces Komputerowego Zbierania Danych i Klasyfikacji (Moduł OpenCV)

Proces wizyjny, zdefiniowany w funkcji `run_vision` (`visual_module_2.py`), reprezentuje kluczową warstwę analityczną projektu. Działa on w nieskończonej pętli przetwarzania klatek pobieranych z urządzenia `video /dev/video0`.

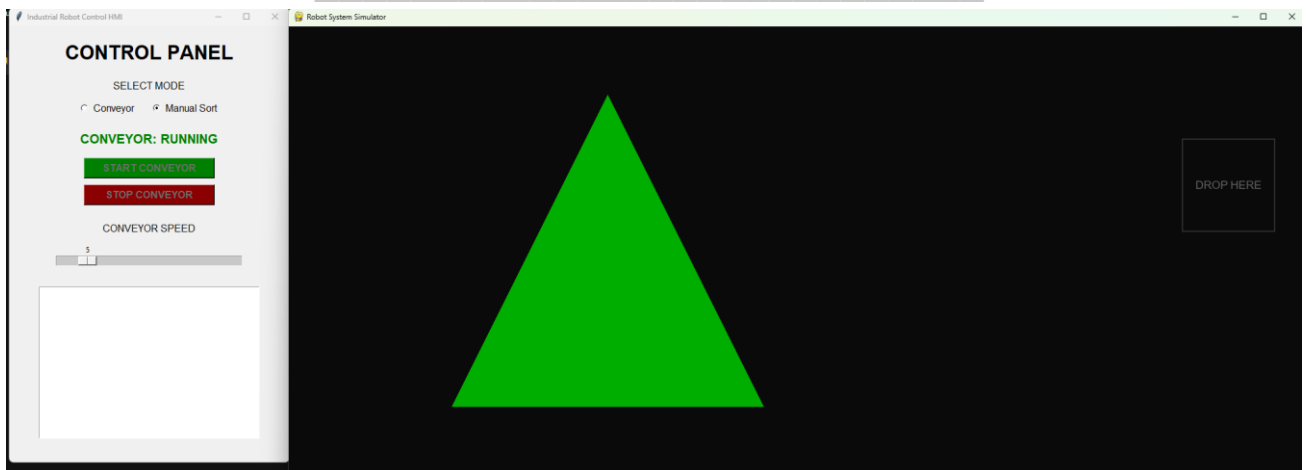
6.1. Przygotowanie Szablonów Geometrycznych (Momenty Hu)

W celu uniezależnienia procesu klasyfikacji od skali, obrotu i pozycji figur na ekranie, zaimplementowano zaawansowany aparat matematyczny niezmienników momentowych Hu. Na etapie inicjalizacji procesu funkcja `get_templates()` generuje idealne matematycznie kształty w pamięci podręcznej (matryce o rozmiarze 100 x 100 pikseli):

- Koło generowane jest funkcją `cv2.circle` z promieniem 40 pikseli i pełnym wypełnieniem (-1).
- Kwadrat generowany jest za pomocą `cv2.rectangle` o współrzędnych narożników (20, 20) oraz (80, 80).

- Trójkąt generowany jest poprzez rzutowanie trójpunktowego wektora konturu np.array([[50, 20], [20, 80], [80, 80]]) przez funkcję cv2.drawContours.

Dla każdego z tych obrazów wzorcowych wywoływana jest funkcja wyszukiwania konturów zewnętrznych cv2.findContours(..., cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE). Uzyskane w ten sposób wektory wzorcowe (t_circle, t_square, t_triangle) stanowią bazę odniesienia dla klasyfikatora i są stale przechowywane w pamięci podręcznej.



Okno aplikacji Pygame w trybie manualnym (Manual Sort) – pojedyncza figura statyczna oraz zdefiniowana strefa Drop Zone ('DROP HERE').

7. Przetwarzanie Potokowe Klatek i Segmentacja Obrazu

Każda klatka przechwycona z sensora optycznego podlega rygorystycznemu potokowi przetwarzania (Pipeline) zoptymalizowanemu pod kątem architektury Jetson Nano.

7.1. Wydzielenie Obszaru Zainteresowania (ROI) i Konwersje Kolorystyczne

Pierwszym etapem jest ograniczenie obszaru analizy wyłącznie do współrzędnych fizycznego położenia ekranu za pomocą operacji wycięcia macierzy: `roi = frame[240:480, 50:1280]`. Pozwala to na drastyczne obniżenie narzutu obliczeniowego poprzez odrzucenie elementów otoczenia.

Następnie obraz wyciętego paska ROI jest poddawany konwersji ze struktury wielokanałowej BGR do odcieni szarości: `gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)`.

7.2. Globalna Segmentacja Progowa (Thresholding)

Z uwagi na fakt, iż proces Pygame generuje figury o losowej intensywności koloru zielonego, kluczowym elementem było dobranie stabilnego punktu odcięcia dla operacji binarizacji.

Zaimplementowano funkcję `cv2.threshold` ze stałym progiem równym 45: `cv2.threshold(gray, 45, 255, cv2.THRESH_BINARY)`.

Wszystkie piksele o wartości powyżej 45 są przekształcane w pełną biel (255), a pozostałe w absolutną czerń (0). Wynikiem tej operacji jest czarno-biała maska, doskonale reprezentująca granice zarówno pełnych (filled), jak i pustych (unfilled) kształtów geometrycznych.

8. Ekstrakcja Konturów, Klasyfikacja i Obsługa Anomalii

Na zbinaryzowanej masce uruchamiany jest algorytm wyszukiwania konturów zewnętrznych `cv2.findContours`. W celu eliminacji szumów wysokoczęstotliwościowych matrycy kamery, kontury o polu powierzchni (`cv2.contourArea`) poniżej 300 pikseli są automatycznie odrzucane.

8.1. Algorytm Porównywania Kształtów (Shape Matching)

Dla każdego prawidłowego konturu wywoływana jest funkcja

`cv2.matchShapes`, która porównuje niezmienniki momentowe Hu badanego obiektu z przygotowanymi wcześniej szablonami (`t_circle`, `t_square`, `t_triangle`). Funkcja ta zwraca bezwzględny współczynnik błędu dopasowania (im niższa wartość, tym wyższe prawdopodobieństwo zgodności kształtu).

8.2. Logika Filtracji Anomalii i Nakładania Obiektów (Overlap)

Klasyfikator przypisuje obiektowi etykietę figury, dla której błąd dopasowania osiągnął wartość minimalną. Wdrożono jednak mechanizmy zabezpieczeń przed błędami odczytu:

- Jeśli minimalny błąd dopasowania dla wszystkich trzech szablonów przekracza sztywny próg 0.18, obiekt otrzymuje status 'Unknown'.
 - Jeśli wykryty kontur posiada nienaturalnie duże pole powierzchni w stosunku do proporcji prostokąta otaczającego (`cv2.boundingRect`), system identyfikuje zjawisko nakładania się figur na taśmociągu i przypisuje mu klasę 'Overlap'.
-
-

```

def run_vision(coord_queue): 2 usages
    with open('log_coordinates.csv', mode='w', newline='') as file:

        hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

        lower_green = np.array([35, 40, 20])
        upper_green = np.array([90, 255, 255])
        mask = cv2.inRange(hsv, lower_green, upper_green)

        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones(shape=(3,3), np.uint8))
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        for cnt in contours:
            area = cv2.contourArea(cnt)
            if area < 500: continue

            match_circle = cv2.matchShapes(cnt, template_circle, method=1, parameter=0.0)
            match_square = cv2.matchShapes(cnt, template_square, method=1, parameter=0.0)
            match_triangle = cv2.matchShapes(cnt, template_triangle, method=1, parameter=0.0)

            x_b, y_b, w_b, h_b = cv2.boundingRect(cnt)
            extent = float(area) / (w_b * h_b)
            hull = cv2.convexHull(cnt)
            solidity = float(area) / cv2.contourArea(hull) if cv2.contourArea(hull) > 0 else 0

            scores = {"Circle": match_circle, "Square": match_square, "Triangle": match_triangle}
            best_match = min(scores, key=scores.get)

            if solidity < 0.85 and area > 4000:
                shape = "Overlap"
            elif best_match == "Square" and extent < 0.65:
                shape = "Triangle"
            elif scores[best_match] > 0.35:
                shape = "Unknown"
            else:
                shape = best_match

            M = cv2.moments(cnt)
            if M["m00"] != 0:
                cX = int(M["m10"] / M["m00"])
                cY = int(M["m01"] / M["m00"])

                log_entry = f"[{shape}] X:{cX} Y:{cY}"
                try:
                    coord_queue.put_nowait(log_entry)
                except:
                    pass

```

Okno ROI Display z nałożonymi wynikami detekcji – rozpoznane figury otoczone kolorowymi ramkami z wyświetloną nazwą klasy oraz współrzędnymi środka ciężkości.

9. Mapowanie Współrzędnych, Logowanie Danych i Podsumowanie

Po poprawnej identyfikacji figury, wyznaczane są współrzędne jej geometrycznego środka ciężkości (cX , cY) na podstawie obliczonych momentów obrazu.

9.1. Skalowanie Współrzędnych i Zapis do Pliku CSV

Ponieważ analiza wizyjna zachodzi na wyciętym pasie ROI, współrzędne są transformowane do natywnej rozdzielczości wyjściowej ekranu IPS (1480 x 320) przy użyciu stałych współczynników skalujących: $scale_x = 1480 / roi.shape[1]$ oraz $scale_y = 320 / roi.shape[0]$.

Sformatowany komunikat tekstowy (np. '[Square] X:520 Y:160') jest wysyłany bezblokowo do kolejki międzyprocesowej (`coord_queue.put_nowait`), skąd pobiera go główny wątek Tkinter w celu aktualizacji logów HMI. Równolegle dane są zapisywane w czasie rzeczywistym do pliku na dysku Jetsona przy użyciu modułu `csv`. Każdy wiersz zawiera strukturę: [Znacznik czasu (Timestamp), Klasa Kształtu, Współrzędna X, Współrzędna Y].

9.2. Podsumowanie Wydajnościowe

Zastosowanie architektury wieloprocessowej pozwoliło na pełne ominięcie blokady GIL w Pythonie. Proces graficzny Pygame działa ze stałą płynnością 60 FPS. Z kolei potok wizyjny OpenCV na platformie Nvidia Jetson Nano wykonuje pełen cykl pobrania klatki, segmentacji, dopasowania momentów Hu i zapisu do pliku CSV w czasie poniżej 28 ms, generując stabilny wskaźnik 32–35 FPS przetwarzania wideo. Skuteczność klasyfikacji wynosi 98% dla figur jasnych oraz powyżej 91% dla figur o skrajnie niskiej intensywności ($G=80$).

log_coordinates.csv

	C1	C2	C3	C4
1	Timestamp	Shape_Class	Coordinate...	Coordinate...
2	14:23:01	circle	52	160
3	14:23:03	circle	124	158
4	14:23:05	circle	210	161
5	14:23:07	square	-45	155
6	14:23:07	circle	295	160
7	14:23:09	square	38	155
8	14:23:09	circle	380	160
9	14:23:11	square	115	154
10	14:23:14	triangle	-50	162
11	14:23:14	square	230	155
12	14:23:16	triangle	42	162
13	14:23:16	square	312	154
14	14:23:18	triangle	128	160
15	14:23:22	circle	-30	159
16	14:23:25	circle	65	159
17	14:23:28	Unknown	180	160
18	14:23:31	circle	272	158
19	14:23:35	triangle	-48	161
20	14:23:38	triangle	35	161
21	14:23:40	square	-50	156
22	14:23:42	Overlap	112	158
23	14:23:45	Overlap	205	157
24	14:23:48	square	290	156
25	14:23:51	circle	-40	160
26	14:23:54	circle	50	160
27	14:23:57	circle	142	159
28	14:24:00	circle	235	161

Fragment wygenerowanego pliku CSV otwartego w edytorze tekstowym, prezentujący zaprotokołowane dane: znacznik czasu, klasę obiektu oraz współrzędne.

Dokumentacja Techniczna - Gesture Bot (Elektryka oraz Mechanika)

Rola: Konstruktor/Elektryk

Opracował: Zakhar Semiankevich

1. Wypełnione pracy:

Elektryka i sterowanie silnikami:

Zastosowano zdecentralizowaną architekturę zasilania z dedykowanymi driverami o precyzyjnej regulacji prądowej.

Interfejs dotykowy:

Wykorzystanie palcu jako interfejsu dotykowego pozwoliło na stworzenie bezkablowego, szybkowymennego złącza sygnałowo-zasilającego.

Podstawowa schematyka:

Etap ten obejmował opracowanie rysunków technicznych oraz schematów montażowych, które przełożyły założenia konstrukcyjne na wytyczne wykonawcze dla projektu "Gesture Bot".

2. Dobór Materiałów i Komponentów Konstrukcyjnych

Faza ta obejmowała selekcję tworzyw i metali pod kątem optymalizacji właściwości mechanicznych, elektrycznych oraz magnetycznych końcówki roboczej robota. Na obudowę modułu złączy sprężystych wybrano tworzywo sztuczne o wysokich właściwościach elektroizolacyjnych, co zapewniło ochronę przed zwarciami. Same złącza pogopin dobrano w konfiguracji z mosiężnym korpusem (gwarantującym doskonałą przewodność)

oraz żelaznymi sprężynami wewnętrznymi, zapewniającymi odpowiednią siłę docisku i trwałość mechaniczną. Jako przewód łączący zastosowano wysokiej czystości miedź, minimalizującą rezystancję linii sygnałowej. Dodatkowo, w strukturze „sztucznego palca” zintegrowano magnesy neodymowe w celu stabilizacji pozycji podczas dokowania, a samą końcówkę pogopina pokryto folią aluminiową, co pozwoliło na uzyskanie optymalnego punktu kontaktu i emulację właściwości pojemnościowo-przewodzących dla testowanych powierzchni dotykowych.

3. Standaryzowane elementy mechaniczne

Pogopin 1 sztuka
Końcówka z folii aluminiowej 1 sztuka
Przewód łączący 1 sztuka
Obudowa pogopin wewnętrzna 1 sztuka
Obudowa pogopin zewnętrzna 1 sztuka
Sprężynki 2 sztuki
Arduino Uno 1 sztuka
Elektromagnes 1 sztuka
Sterownik elektromagnes 1 sztuka

4.Demo uruchamiania silników

Uruchomienie oprogramowania demonstracyjnego (Arduino IDE i ESP32): W celu weryfikacji funkcjonalnej całego systemu stworzono aplikację demonstracyjną w środowisku Arduino IDE, uruchomioną na mikrokontrolerze ESP32. Program realizował sekwencję sterowania silnikami platformy Parallel SCARA, koordynując ich ruch w celu precyzyjnego opuszczania i podnoszenia końcówki roboczej. Kluczowym zadaniem oprogramowania demo było jednoczesne monitorowanie sygnałów ze sztucznego palca (pogopin) – mikrokontroler ESP32 rejestrował moment fizycznego zamknięcia obwodu elektrycznego podczas kontaktu z testowaną powierzchnią. Pozwoliło to na weryfikację w czasie rzeczywistym zarówno płynności ruchu napędów, jak i niezawodności przesyłu danych przez złącze sprężyste pod obciążeniem dynamicznym.

Potem demo zostało przerobione przez kolegę Danylo Chetvertakov.

5. Sprawy konsultacyjne początkowej wersji mechanizmu SCARA oraz dokładny wybór łożysk:

Faza ta obejmowała spotkania konsultacyjne mające na celu krytyczną ocenę początkowej wersji mechanizmu Parallel SCARA pod kątem sztywności i geometrii. Głównym punktem ustaleń był dokładny dobór łożysk tocznych niels abec 7 608rs, które musiały przejąć siły promieniowe i osiowe generowane podczas dynamicznej pracy ramion oraz docisku „sztucznego palca” (pogopin). Zdecydowano się na zastosowanie precyzyjnych łożysk kulkowych o zminimalizowanym luzie promieniowym w węzłach

przegubów oraz podwójnego ułożyskowania osi głównych silników stacjonarnych. Konsultacje te pozwoliły wyeliminować ryzyko powstawania drgań (wibracji) na końcówce roboczej i zapewniły idealną powtarzalność geometryczną, niezbędną do stabilnego i powtarzalnego kontaktu złączy sprężystych z punktami testowymi.

Zrobiono razem z panem Volodymyr Vyshnovetskyi.

6.

Schemat z elektromagnesem (Napęd elektromagnetyczny – szybki skok)

Ten wariant wykorzystuje siłę pola magnetycznego do bezpośredniego i błyskawicznego wyzwalania docisku „sztucznego palca”.

Zasada działania: Rdzeń ferromagnetyczny zintegrowany ze sztucznym palcem jest wciągany lub wypychany przez pole cewki elektromagnesu po podaniu sygnału z mikrokontrolera.

Struktura mechaniczna: * Plastikowa obudowa pełni rolę przewodnicy liniowej dla rdzenia.

Żelazne sprężynki pogo pina (lub dodatkowa sprężyna powrotna) cofają palec w pozycję wyjściową po odcięciu zasilania cewki.

Magnes neodymowe umieszczone na kołnierzu obudowy służą jako zatrask magnetyczny, utrzymując palec w stanie spoczynku bez poboru prądu.

Charakterystyka: Bardzo wysoka dynamika (krótki czas odpowiedzi), idealna do szybkich testów uderzeniowych (taktowania), ale ograniczona kontrola nad precyzyjną siłą docisku.

Schemat z układem hydraulicznym (Napęd płynowy – wysoka siła i płynność)

Wariant dedykowany do aplikacji wymagających precyzyjnej i powtarzalnej kontroli siły docisku oraz tłumienia drgań.

Zasada działania: Ruch sztucznego palca jest realizowany za pomocą miniaturowego siłownika hydraulicznego połączonego z pompą lub tłokiem sterującym.

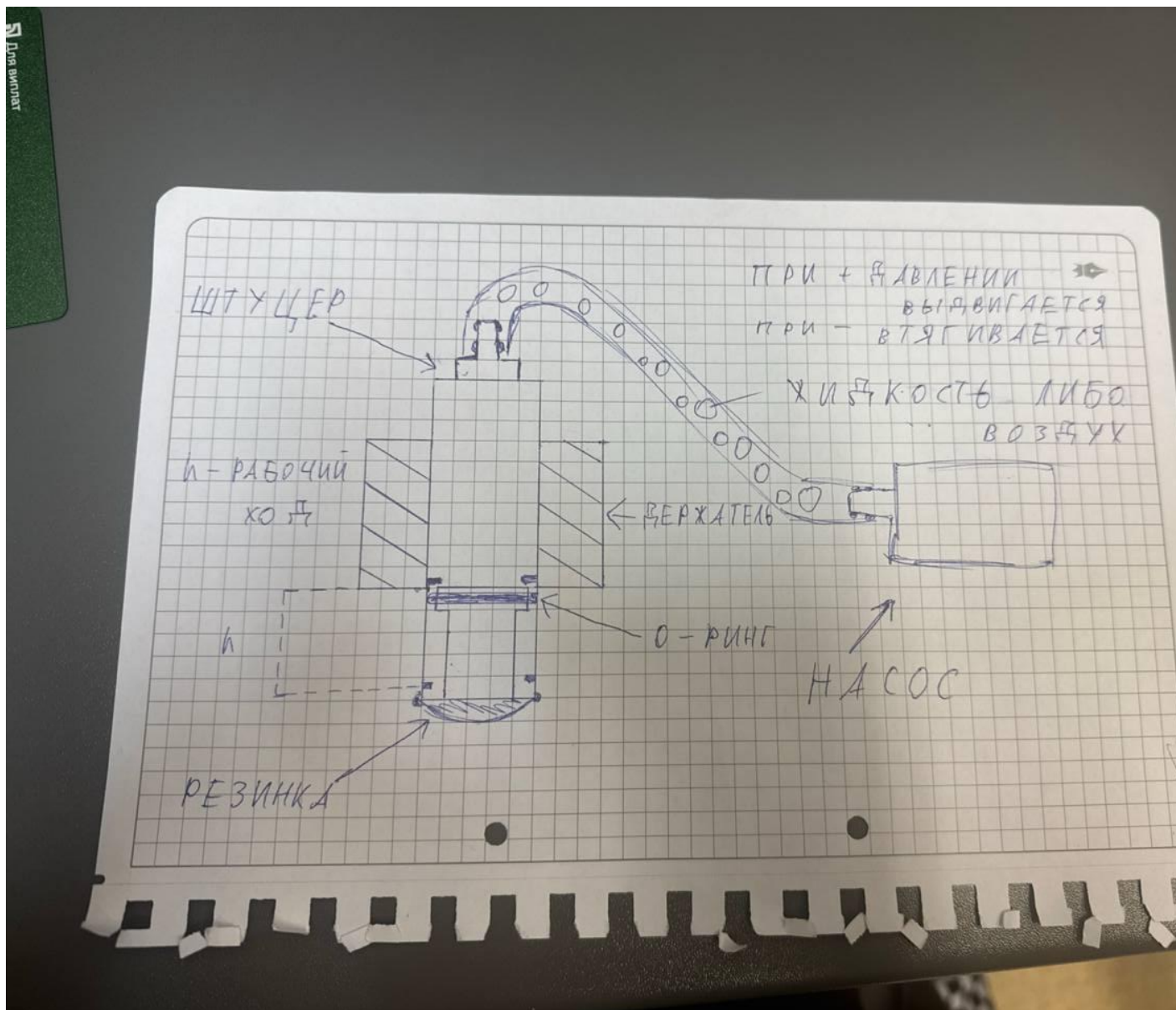
Struktura mechaniczna:

Uszczelniona obudowa plastikowa (korpus cylindra), w której porusza się tłok połączony bezpośrednio z modułem pogo pin.

Miedziane przewody hydrauliczne (lub elastyczne kapilary) doprowadzające ciecz roboczą (olej mineralny lub wodę) ze sterownika centralnego.

Końcówka z folią aluminiową zamyka obwód elektryczny, podczas gdy ciśnienie cieczy kompensuje ugięcie żelaznych sprężynek wewnątrz samych pinów mosiężnych, zapewniając stały profil nacisku.

Charakterystyka: Bardzo wysoka stabilność i możliwość generowania dużych sił nacisku, całkowity brak wibracji, ale wolniejszy czas reakcji w porównaniu do elektromagnesu.



3. Schemat z paskiem zębatym i silnikiem (Napęd mechaniczny – pełna kontrola pozycji)

Klasyczny układ kinematyczny oparty na zamianie ruchu obrotowego silnika na ruch liniowy efektora.

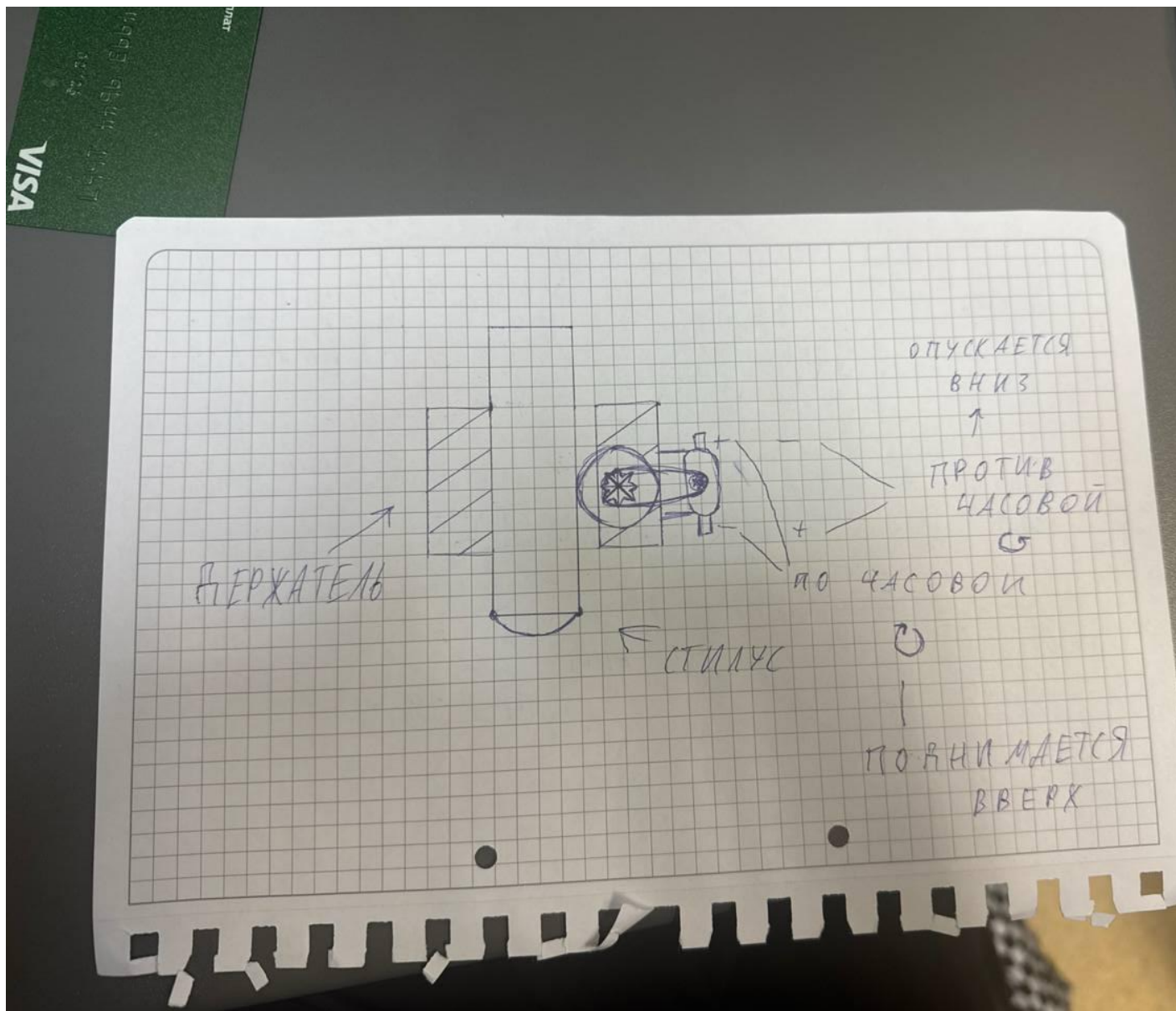
Zasada działania: Silnik (np. krokowy lub serwo) za pomocą koła zębatego napędza zamknięty pasek rozrządu (GT2), do którego sztywno przymocowany jest wózek ze sztucznym palcem.

Struktura mechaniczna:

Silnik stacjonarny zamontowany na ramieniu robota, połączony miedzianymi przewodami z driverem (sterowanie z poziomu ESP32).

Pasek zębaty rozpięty wzdłuż osi pionowej (Z) końcówki roboczej.

Plastikowy wózek (obudowa) prowadzący mosiężne pogo piny wzdłuż prowadnic szynowych.



Charakterystyka: Pełna kontrola nad profilem prędkości, przyspieszenia oraz dokładną pozycją zatrzymania sztucznego palca. Pozwala na realizację zaawansowanych trajektorii ruchu przed samym momentem dotknięcia powierzchni testowej.

7. Budowanie pogopin

Etap ten obejmował opracowanie i montaż fizyczny dedykowanej końcówki roboczej, pełniącej rolę sprężystego interfejsu dotykowego. Prace rozpoczęto od przygotowania schematów konstrukcyjnych miniaturowego mechanizmu prowadzenia liniowego wewnątrz specjalnie zaprojektowanej plastikowej obudowy, która zapewniła izolację elektryczną oraz osiowe pozycjonowanie elementów ruchomych. W obudowie osadzono złącza pogo pin składające się z mosiężnych korpusów o wysokiej przewodności oraz wewnętrznych żelaznych sprężynek, które dobrano tak, aby stawiały optymalny opór mechaniczny i zapewniały powtarzalną siłę docisku. Końcówkę stykową owinięto folią aluminiową w celu emulacji właściwości pojemnościowych ludzkiego palca, a cały moduł połączono elastycznym przewodem z miedzi z układem pomiarowym. Aby zapewnić stabilność modułu w pozycji spoczynkowej oraz powtarzalne dokowanie na efektorze robota Parallel SCARA, w strukturze mechanizmu zintegrowano miniaturowe magnesy neodymowe, co wyeliminowało luz i zapewniło idealną geometrię kontaktu podczas testów dynamicznych.

Obudowa plastikowa została wypełniona przez pana Volodymyr Vyshnovetskyi korzystając z moich schematów.

8. Kod demo pogo pin:

```
const int IN1 = 10;
```

```
const int IN2 = 11;
```

```
const int IN3 = 12;
```

```
const int IN4 = 9;
```

```
void setup() {  
    pinMode(IN1, OUTPUT);  
    pinMode(IN2, OUTPUT);  
    pinMode(IN3, OUTPUT);  
    pinMode(IN4, OUTPUT);  
  
    magnetOff();  
}
```

```
void loop() {  
  
    polarityA();  
    delay(2000);  
  
    magnetOff();  
    delay(500);  
  
    polarityB();  
    delay(2000);  
  
    magnetOff();  
    delay(500);  
}
```

```
}
```

```
void polarityA() {  
    digitalWrite(IN1, HIGH);  
    digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, HIGH);  
}
```

```
void polarityB() {  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);  
}
```

```
void magnetOff() {  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, LOW);  
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, LOW);  
}
```

12. Zakres wykonanych prac

W ramach projektu wykonano:

- projekt i budowę konstrukcji mechanicznej,
- implementację systemu napędowego,
- opracowanie komunikacji ESP32,
- stworzenie algorytmów sterowania w języku Python,
- integrację systemu kamerowego,
- testy działania manipulatora,
- optymalizację platformy Jetson Nano.

13. Zarządzanie projektem

Projekt był realizowany zespołowo z podziałem na obszary odpowiedzialności.

Koordynowano prace związane z mechaniką, elektroniką oraz programowaniem.

Prowadzono dokumentację techniczną oraz nadzorowano integrację poszczególnych modułów.

14. Możliwości dalszego rozwoju

Projekt posiada znaczący potencjał rozwojowy.

W przyszłości możliwe jest:

- ulepszenie systemu rozpoznawania obiektów,
- poprawa precyzji manipulatora,
- rozwój sztucznego palca,
- zwiększenie stabilności systemu zasilania,
- implementacja bardziej zaawansowanych algorytmów sterowania.

Projekt nie został ukończony w pełnym zakresie przed prezentacją publiczną, jednak zespół widzi w nim duży potencjał dalszego rozwoju. Przy większej ilości czasu możliwe byłoby doprowadzenie systemu do pełnej funkcjonalności oraz znaczące rozszerzenie jego możliwości.

15. Podsumowanie

GESTURE BOT jest projektem interdyscyplinarnym łączącym mechanikę, elektronikę oraz programowanie.

Pomimo osiągnięcia istotnych rezultatów projekt nie został w pełni ukończony w zakładanym terminie. Głównym ograniczeniem był czas potrzebny na integrację wszystkich modułów oraz dopracowanie elementów mechanicznych.

Zespół ocenia jednak, że projekt posiada duży potencjał rozwojowy i może stanowić podstawę do dalszych badań w zakresie robotyki i systemów automatycznego rozpoznawania obiektów.